HACETTEPE UNIVERSITY

ELECTRICAL AND ELECTRONICS ENGINEERING

ELE432 - Advanced Digital Design

PRELIMINARY WORK III - Multicycle Processor

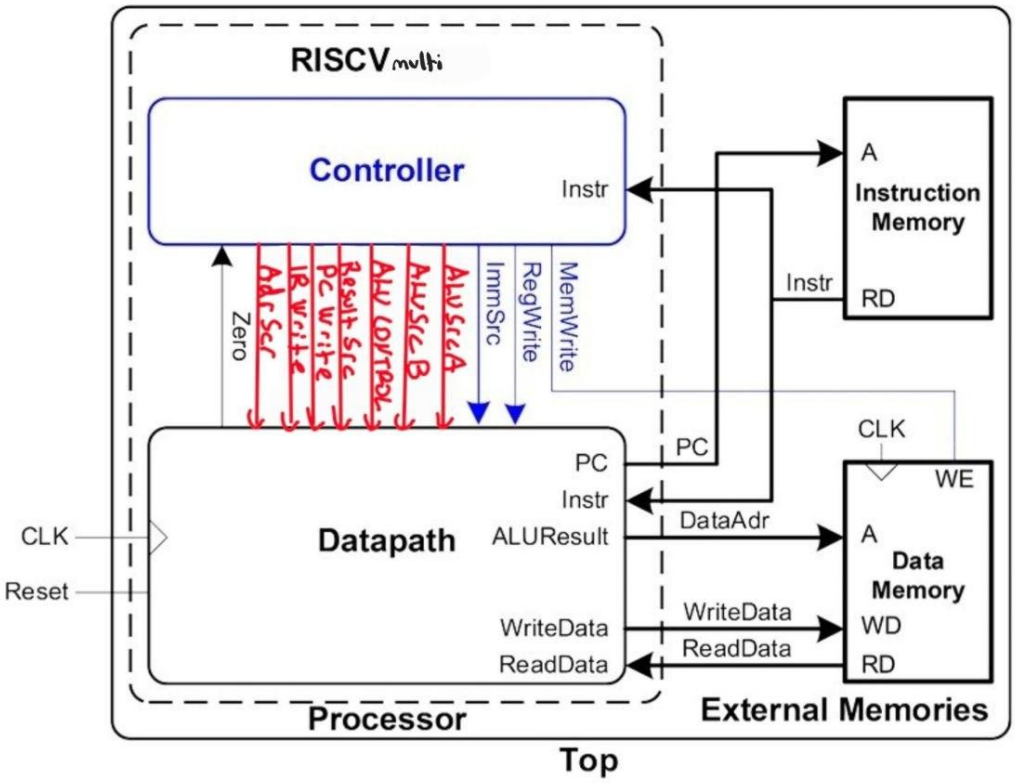
Name & Surname: BERKE MALÇOK

Student ID: 2210357081

1)

The total time spent on this lab is approximately 4 hours. Since the single-cycle processor and the controller design were already completed in Lab 2 and Homework 2, the implementation of the multicycle processor took less time.

2)



3)

The design follows a hierarchical structure as shown below:

```
Top (top.sv)
├── riscvmulti (riscvmulti.sv)
│   ├── controller (controller.sv)
│   │   ├── maindec (maindec.sv)
│   │   ├── aludec (aludec.sv)
│   │   └── instrdec (instrdec.sv)
│   └── datapath (datapath.sv)
│       ├── alu (alu.sv)
│       ├── regfile (regfile.sv)
│       ├── extend (extend.sv)
│       ├── flopr
│       └── mux
└── mem (mem.sv)
```

top.sv

```
module top(
   input  logic       clk, reset,
   output logic [31:0] WriteData, DataAdr,
   output logic       MemWrite
);

   logic [31:0] ReadData;

   riscvmulti rvmulti(
      .clk(clk),
      .reset(reset),
      .MemWrite(MemWrite),
      .Adr(DataAdr),
      .WriteData(WriteData),
      .ReadData(ReadData)
   );

   mem mem1(
      .clk(clk),
      .we(MemWrite),
      .a(DataAdr),
      .wd(WriteData),
      .rd(ReadData)
   );

endmodule
```

Riscvmulti.sv

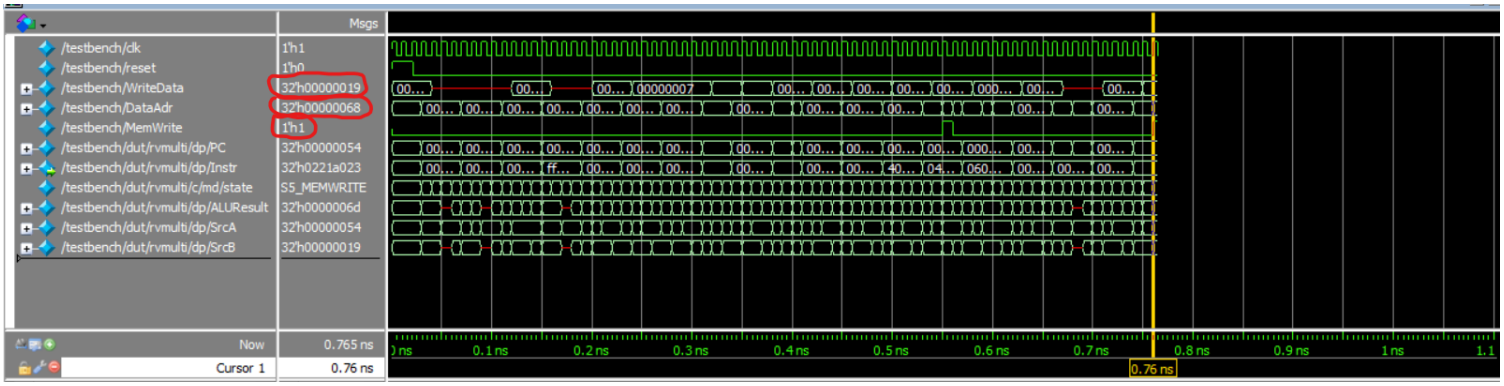
```
module riscvmulti(
   input  logic       clk, reset,
   output logic       MemWrite,
   output logic [31:0] Adr, WriteData,
   input  logic [31:0] ReadData
);

   logic [6:0] op;
   logic [2:0] funct3;
   logic       funct7b5;
   logic       Zero;

   logic [1:0] ImmSrc;
   logic [1:0] ALUSrcA, ALUSrcB;
   logic [1:0] ResultSrc;
   logic       AdrSrc;
   logic [2:0] ALUControl;
   logic       IRWrite, PCWrite, RegWrite;

   logic [31:0] Instr;

   controller c(
      .clk(clk),
      .reset(reset),
      .op(op),
      .funct3(funct3),
      .funct7b5(funct7b5),
      .zero(Zero),
      .immsrc(ImmSrc),
      .alusrca(ALUSrcA),
      .alusrcb(ALUSrcB),
      .resultsrc(ResultSrc),
      .adrsrc(AdrSrc),
      .alucontrol(ALUControl),
      .irwrite(IRWrite),
      .pcwrite(PCWrite),
      .regwrite(RegWrite),
      .memwrite(MemWrite)
   );

   datapath dp(
      .clk(clk),
      .reset(reset),
      .ImmSrc(ImmSrc),
      .ALUSrcA(ALUSrcA),
      .ALUSrcB(ALUSrcB),
      .ResultSrc(ResultSrc),
      .AdrSrc(AdrSrc),
      .ALUControl(ALUControl),
      .IRWrite(IRWrite),
      .PCWrite(PCWrite),
      .RegWrite(RegWrite),
      .ReadData(ReadData),
      .Zero(Zero),
      .Adr(Adr),
      .WriteData(WriteData),
      .Instr(Instr)
   );

   assign op      = Instr[6:0];
   assign funct3   = Instr[14:12];
   assign funct7b5 = Instr[30];

endmodule
```

4)

Step	PC	Instr	State	Result	Result Notes
3	0x00	0x00500113	S0: Fetch	4	PC + 4
4	0x04	0x00500113	S1: Decode	X	OldPC + immediate
5	0x04	0x00500113	S8: ExecuteI	X	ALUResult = x0 (0) + 5 = 5
6	0x04	0x00500113	S7: ALUWB	5	Result=ALUOUT
7	0x04	0x00C00193	Fetch	8	PC + 4
8	0x08	0x00C00193	S1: Decode	X	OldPC + immediate
9	0x08	0x00C00193	S8: ExecuteI	X	ALUResult = x0 (0) + 12 = 12
10	0x08	0x00C00193	S7: ALUWB	12	Result = ALUOUT
11	0x08	0x0021C233	Fetch	12	PC + 4
12	0x0C	0x0021C233	S1: Decode	X	Register read
13	0x0C	0x0021C233	S6: ExecuteR	X	ALUResult = 12 xor 5 = 9
14	0x0C	0x0021C233	S7: ALUWB	9	Result = ALUOUT

5)

The waveform shows that the processor successfully passes the testbench.
When MemWrite is asserted, the value 25 (0x19) is written to memory address 104 (0x68).
This confirms that the processor executes instructions correctly and produces the expected result.



6)

github link : https://github.com/berkeee112/riscv-multicycle-lab3.git